

Cilium Architecture & Kube-Proxy Migration

eBPF Kernel Routing, OSI Layer Mapping, WebSocket Optimizations, and Migration Playbook

1. What is Cilium?

Cilium is an open-source, high-performance Networking, Security, and Observability solution for Kubernetes. Powered by **eBPF (Extended Berkeley Packet Filter)**, Cilium injects dynamic networking and security logic directly inside the Linux kernel space.

2. Where to Use Cilium?

- **Production Kubernetes Clusters:** As a modern CNI replacing legacy kube-proxy.
- **Multi-Cloud & Hybrid Deployments:** Unifying networking across AWS (EKS), GCP (GKE), Azure (AKS), and Bare-Metal / OVHcloud MKS.
- **High-Scale Microservices:** Environments with hundreds of Pods where iptables causes packet evaluation bottlenecks.
- **Zero-Trust Frameworks:** Enforcing label-based security at L3/L4 and fine-grained L7 application rules.

3. What Problem Does Cilium Solve?

Standard Kubernetes uses kube-proxy to populate sequential iptables chains. As clusters grow, every packet must evaluate thousands of rules one-by-one. Cilium bypasses iptables entirely using eBPF **hash-table lookups** in kernel memory, achieving constant $O(1)$ routing performance regardless of cluster size.

4. What Cilium Offers Beyond Standard CNI

Capability	Standard K8s (Kube-Proxy + Iptables)	Cilium (eBPF Engine)
Routing Performance	$O(N)$ sequential list evaluation	$O(1)$ instantaneous hash-table lookup
Security Paradigm	Brittle IP-based rules	Stable, identity-based security via Pod labels
Multi-Layer Firewall	Layer 3 and Layer 4 only	Layer 3, Layer 4, and selective Layer 7
Observability	Requires heavy external sidecars	Native, kernel-level flow tracking via Hubble

5. How the Manifest File Unlocks OSI Layers

In a CiliumNetworkPolicy, each progressive indentation unlocks a deeper layer of the network packet:

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
spec:
  endpointSelector:
    matchLabels:
      app: backend-server
  ingress:
    - fromEndpoints:
      - matchLabels:
          app: frontend      # UNLOCKS LAYER 3 (Pod Identity / IP)
        toPorts:
          - ports:
              - port: "8080"  # UNLOCKS LAYER 4 (Transport / TCP Port)
                protocol: TCP
            rules:             # UNLOCKS LAYER 7 (Application / Envoy Proxy)
```

```
http:
- method: "GET"
  path: "/api"
```

6. Why WebSockets Should Stick to Layer 4 (Max)

The Layer 7 Envoy Proxy Trap for WebSockets

Defining a rules: block (Layer 7) triggers Cilium to redirect traffic to an embedded **Envoy Proxy** sidecar. Envoy is optimized for short HTTP request-response cycles. WebSockets start as HTTP but upgrade to persistent TCP tunnels; Envoy's default timeouts often terminate these silent channels (1006 drops).

The Fix: Keep WebSocket policies at **Layer 4** (Port/TCP only). Traffic stays inside pure eBPF kernels as raw TCP packets, avoiding Envoy interference.

7. Multi-Policy Architecture & Distributed Agent Translation

Multiple CiliumNetworkPolicy manifests submitted across namespaces are aggregated by the Kubernetes API Server and distributed to every node. A local cilium-agent (DaemonSet) compiles them into optimized eBPF bytecode and loads them directly into the node's kernel for decentralized enforcement.

8. Step-by-Step Migration Guide: Kube-Proxy to Cilium

Migrating an existing cluster from kube-proxy to a fully ready eBPF Cilium setup (`kubeProxyReplacement=true`) involves a zero-downtime, staged approach.

Phase 1: Pre-requisites & Environment Audit

- **Linux Kernel Version:** Ensure all worker nodes run Kernel ≥ 5.4 (5.15+ recommended for full eBPF socket-lb features).
- **K8s API Server Endpoint:** Retrieve the explicit internal IP and port of your control plane API Server (`KUBE\_APISERVER\_IP` and `KUBE\_APISERVER\_PORT`).

Phase 2: Deploy Cilium with Kube-Proxy Replacement

Deploy Cilium via Helm using the `kubeProxyReplacement=true` flag so eBPF handles all service routing and load balancing:

```
helm repo add cilium https://helm.cilium.io/
helm repo update

helm install cilium cilium/cilium --namespace kube-system --set kubeProxyReplacement=true --set
k8sServiceHost=$KUBE_APISERVER_IP --set k8sServicePort=$KUBE_APISERVER_PORT --set
operator.replicas=2
```

Phase 3: Disable & Remove Legacy Kube-Proxy

Once Cilium pods are running successfully, disable the default kube-proxy DaemonSet to stop iptables rule generation:

```
# Stop kube-proxy from restarting
kubectl -n kube-system patch daemonset kube-proxy -p '{"spec":{"template":{"spec":{"nodeSelector":
{"non-existing": "true"}}}}}'

# Delete the legacy kube-proxy DaemonSet
kubectl -n kube-system delete daemonset kube-proxy
kubectl -n kube-system delete configmap kube-proxy
```

Phase 4: Purge Legacy Iptables Rules

Flush leftover iptables rules created by kube-proxy across worker nodes (can be executed via a DaemonSet or node SSH):

```
# Run on worker nodes or via a cleanup Job
iptables-save | grep -v KUBE | iptables-restore
iptables -F -t nat
iptables -F -t filter
```

Phase 5: Validation & Cluster Readiness Verification

Confirm Cilium has completely taken over `kube-proxy` functionality across all nodes:

```
# 1. Check overall Cilium health
cilium status --wait

# 2. Verify KubeProxyReplacement status inside the agent pod
kubectl -n kube-system exec ds/cilium -- cilium-dbg status | grep KubeProxyReplacement
# Output must show: KubeProxyReplacement: Strict (or True)

# 3. Run full connectivity and network policy tests
cilium connectivity test
```

Migration Complete: Your cluster is now fully operating with Cilium eBPF packet routing, zero iptables overhead, and high-performance L3-L7 policy capabilities.